

Universidad Nacional Autónoma de México

Facultad de Ciencias

Tarea 04

No. de cuenta	Nombre
320148204	González Lucero Alan Uriel
321225087	Rivera Jiménez Isaac
321306102	Tapia Sánchez Jesús



Complejidad Computacional

Semestre 2027-1

Índice

1 Ejercicio 1 **2**

2 Ejercicio 2 **4**

3 Ejercicio 3 **11**

4 Ejercicio 4 **12**

Ejercicio 1

Enunciado

Sean L_1, L_2 definidos sobre Σ^*. Definimos la operación $\oplus$ de la siguiente manera:

$$L_1 \oplus L_2 = \{x \mid x \text{ está en exactamente uno de los lenguajes, en } L_1 \text{ o (exclusivo) en } L_2\}$$

Supón que ambos lenguajes, L_1, L_2, pertenecen a las clases de complejidad NP y coNP. Prueba que $L_1 \oplus L_2 \in \text{NP} \cap \text{coNP}$.

Solución

Por definición de $L_1 \oplus L_2$, sabemos que las cadenas x que contiene dicho conjunto cumplen con que x sólo puede pertenecer en L_1 o L_2, no en ambos.
Sea x una cadena y $x \in L_1 \oplus L_2$ entonces tenemos los siguientes casos:
Caso 1 | $x \in L_1$
Si $x \in L_1$ por def. de $L_1 \oplus L_2$ tenemos que $x \notin L_2$.
De lo cual, por def. del complemento de un lenguaje:

$$\bar{L_2} = \Sigma^* - L_2 = \{x \mid x \in \Sigma^* \text{ y } x \notin L_2\} \Rightarrow \text{en particular } x \in \bar{L_2} \Rightarrow x \in L_1 \cap \bar{L_2}$$

Caso 2 | $x \in L_2$
Si $x \in L_2 \Rightarrow x \notin L_1 \Rightarrow x \in \bar{L_1} \Rightarrow x \in L_2 \cap \bar{L_1}$

C.O.C. | $x \in (L_1 \cap \bar{L_2}) \cup (L_2 \cap \bar{L_1})$
Por lo que, podemos concluir que $L_1 \oplus L_2 = (L_1 \cap \bar{L_2}) \cup (L_2 \cap \bar{L_1})$.

Por otra parte, el problema nos pide suponer que ambos lenguajes L_1, L_2 pertenecen a NP y coNP. Es decir:

$$L_1, L_2 \in \text{NP} \cap \text{coNP}$$

Entonces, por def. de intersección:

$$L_1, L_2 \in \text{NP} \text{ y } L_1, L_2 \in \text{coNP}$$

Como $L_1, L_2 \in \text{NP} \Rightarrow$ por def. de coNP $\bar{L_1}, \bar{L_2} \in \text{coNP}$.
Como $L_1, L_2 \in \text{coNP} \Rightarrow$ por def. de coNP $\bar{L_1}, \bar{L_2} \in \text{NP}$.
$\therefore \bar{L_1}, \bar{L_2} \in \text{NP} \cap \text{coNP}$.

Ahora bien, después de todo este análisis. Para demostrar que $L_1 \oplus L_2 \in \text{NP} \cap \text{coNP}$, tendremos que probar que $L_1 \oplus L_2 \in \text{NP}$ y $L_1 \oplus L_2 \in \text{coNP}$.

P.D. que $L_1 \veebar L_2 \in NP$.

$$L_1 \veebar L_2 = (L_1 \wedge \overline{L_2}) \vee (L_2 \wedge \overline{L_1})$$

Por las anteriores demostraciones, sabemos que $L_1 \in NP$ y $\overline{L_2} \in NP \Rightarrow$ aprovechando la cerradura bajo la intersección de NP $A = (L_1 \wedge \overline{L_2}) \in NP$.
De manera análoga, como $L_2 \in NP$ y $\overline{L_1} \in NP \Rightarrow B = (L_2 \wedge \overline{L_1}) \in NP$.

Como $A \in NP$ y $B \in NP \Rightarrow$ por la cerradura bajo la unión de NP $A \cup B \in NP$.
Por lo tanto:

$$L_1 \veebar L_2 = (L_1 \wedge \overline{L_2}) \vee (L_2 \wedge \overline{L_1}) \in NP \quad //$$

P.D. que $L_1 \veebar L_2 \in coNP$.
Para probar que un lenguaje pertenece a coNP, basta con demostrar que su complemento pertenece a NP.
Calculamos el complemento de $L_1 \veebar L_2$ utilizando las leyes de De Morgan:

$$\overline{L_1 \veebar L_2} = (L_1 \wedge \overline{L_2}) \vee (L_2 \wedge \overline{L_1})$$
$$= \overline{(L_1 \wedge \overline{L_2})} \wedge \overline{(L_2 \wedge \overline{L_1})}$$
$$= (\overline{L_1} \vee L_2) \wedge (\overline{L_2} \vee L_1)$$

Distribuyendo los términos:

$$\overline{L_1 \veebar L_2} = (\overline{L_1} \wedge \overline{L_2}) \vee \cancel{(\overline{L_1} \wedge L_1)^{\perp}} \vee \cancel{(L_2 \wedge \overline{L_2})^{\perp}} \vee (L_2 \wedge L_1)$$

Como $(\overline{L_1} \wedge L_1) = \perp$ y $(L_2 \wedge \overline{L_2}) = \perp$, simplificamos a:

$$\overline{L_1 \veebar L_2} = (L_1 \wedge L_2) \vee (\overline{L_1} \wedge \overline{L_2})$$

Por otro lado, sabemos que:

$$L_1, L_2 \in NP \Rightarrow (L_1 \wedge L_2) \in NP$$
$$\overline{L_1}, \overline{L_2} \in NP \Rightarrow (\overline{L_1} \wedge \overline{L_2}) \in NP$$

Entonces, utilizando la cerradura bajo la unión de NP:

$$(L_1 \wedge L_2) \vee (\overline{L_1} \wedge \overline{L_2}) \in NP \Rightarrow \overline{L_1 \veebar L_2} \in NP$$

$\therefore$ al ser el complemento un lenguaje en NP:

$$L_1 \veebar L_2 \in coNP \quad //$$

Conclusión.
Hemos demostrado que $L_1 \veebar L_2 \in NP$ y $L_1 \veebar L_2 \in coNP$, en particular:

$$L_1 \veebar L_2 \in NP \cap coNP$$

Ejercicio 2

Enunciado

¿Cuáles de los siguientes problemas pertenecen a P, NP ó coNP? ¿Cuáles a coNP? Justifica tu respuesta argumentando tu conjetura o dando la demostración de pertenencia a las clases correspondientes.
Nota: Considerar que al menos una demostración de pertenencia a NP se deberá realizar utilizando la noción de No Determinismo, y al menos una utilizando la definición alternativa (con la noción de verificación).

- **DOM** $= \{\langle G, K\rangle \mid G$ tiene un conjunto dominante de K nodos$\}$
  Un subconjunto de vértices C de G es un conjunto dominante de G, si todo otro vértice de G es adyacente a algún vértice de C.

Solución

Definición Formal del Lenguaje DOM

El lenguaje del problema del conjunto dominante en su versión de decisión se define como:

$\text{DOM} = \{\langle G, K\rangle \mid G$ es una gráfica no dirigida y G tiene un conjunto dominante de K nodos$\}$

Un subconjunto de vértices $C \subseteq V(G)$ es un **conjunto dominante** si $|C| \leq K$ y para todo vértice $v \in V(G) \setminus C$, existe al menos un vértice $u \in C$ tal que $\{u, v\} \in E(G)$.

Definición de NP por Verificación

Un lenguaje L pertenece a la clase **NP** si existen un polinomio $p(n)$ y una Máquina de Turing determinista M de tiempo polinomial tales que para toda cadena de entrada x:

$$x \in L \iff \exists u \in \{0,1\}^{p(|x|)} \text{ tal que } M(x, u) = 1$$

P.D. que el problema DOM pertenece a NP mediante un verificador determinista

1. Definición del Certificado (Testigo) u y Esquema de Codificación

- **Certificado (u):** Es el subconjunto candidato de vértices $C = \{v_1, v_2, \ldots, v_m\} \subseteq V(G)$.
- **Esquema de Codificación:** El certificado se codifica como una lista explícita de los índices binarios de los vértices seleccionados o como un vector binario de longitud $|V|$ sobre el alfabeto $\{0, 1\}$.
- **Cota Polinomial:** El tamaño del certificado cumple $|u| \leq O(|V| \log |V|)$, el cual está acotado por un polinomio $p(|x|)$ respecto a la longitud de la representación binaria de la entrada $x = |\langle G, K\rangle|$.

2. Algoritmo del Verificador Determinista $M(x, u)$

La máquina de Turing determinista M recibe la pareja de entrada $x = (G, K)$ y el certificado candidato $u = C$ y ejecuta la siguiente secuencia de pasos:

1. Calcula la cardinalidad del conjunto C. Si $|C| > K$ o si C contiene elementos fuera de $V(G)$, la máquina **rechaza** inmediatamente devolviendo `0`.
2. Inicializa un conjunto de nodos cubiertos $S = C$.
3. Para cada vértice $u \in C$, recorre sus listas de adyacencia en la gráfica G y agrega todos sus vecinos inmediatos al conjunto S.
4. Compara la cantidad de nodos en S con el total de vértices $|V(G)|$. Si $S = V(G)$ (es decir, todo nodo $v \notin C$ tiene al menos un vecino en C), la máquina **acepta** devolviendo `1`; de lo contrario, **rechaza** devolviendo `0`.

3. Análisis de Complejidad en Tiempo

– El análisis de cardinalidad y validez en el Paso 1 toma tiempo $O(|V|)$.

– El recorrido de los vértices de C y el marcado de sus vecinos en los Pasos 2 y 3 toma un tiempo proporcional al grado de los nodos examinados, acotado a lo más por el número de aristas de la gráfica $O(|V| + |E|)$.

– El Paso 4 consiste en comparar el número de nodos marcados con $|V|$, lo cual toma tiempo $O(|V|)$.

El tiempo total de ejecución del verificador M está acotado por $O(|V| + |E|)$ operaciones deterministas, lo cual representa una función polinomial en el tamaño del ejemplar de entrada $|\langle G, K\rangle|$.

Conclusión

Si $(G, K) \in$ DOM, existe al menos un subconjunto de vértices C con $|C| \leq K$ que domina a G. Dicho conjunto sirve como certificado u de tamaño polinomial que hace que el verificador responda $M((G, K), C) = 1$.

Si $(G, K) \notin$ DOM, no existe ningún conjunto de tamaño $\leq K$ capaz de dominar a G. Por ende, para cualquier certificado candidato u, el verificador responderá $M((G, K), u) = 0$.

Dado que se definió un certificado de tamaño polinomial y un verificador determinista de tiempo polinomial, queda demostrado que DOM $\in$ **NP**.

- **L** $= \{\langle G\rangle \mid G$ es una gráfica no dirigida que NO contiene ciclos Eulerianos$\}$

Solución

Definición Formal del Lenguaje L

El lenguaje del problema se define como:

$$L = \{\langle G \rangle \mid G \text{ es una gráfica no dirigida que NO contiene ciclos Eulerianos}\}$$

Un **ciclo Euleriano** en una gráfica no dirigida es un ciclo que recorre cada arista de G exactamente una vez.

Teorema de Euler

Por el **Teorema de Euler**:

> Una gráfica conexa G contiene un ciclo Euleriano si y solo si el grado de todos sus vértices es **par**.

Por lo tanto, su condición de negación lógica establece que una gráfica G **NO contiene un ciclo Euleriano** si y solo si cumple al menos una de las siguientes dos condiciones:

1. Existe al menos un vértice $v \in V(G)$ cuyo **grado es impar** ($\text{grado}(v) \pmod 2 \neq 0$).
2. La gráfica G es **desconexa** (sus vértices no aislados están divididos en dos o más componentes conexas distintas).

Clasificación del Problema

Gracias al teorema de Euler, el lenguaje L **pertenece a la clase P**, ya que es posible decidir determinísticamente si una gráfica no contiene ciclos Eulerianos mediante un algoritmo de tiempo polinomial.

P.D. que L pertenece a P mediante una máquina determinista decidible

1. Algoritmo Determinista de Decisión $M(\langle G \rangle)$

Dada la codificación de una gráfica no dirigida $G = (V, E)$:

- Recorre la lista de adyacencia o matriz de adyacencia de G y calcula el grado de cada vértice $v \in V$.
- Si encuentra algún vértice v con **grado impar**, la máquina **acepta** inmediatamente (devuelve `1`), confirmando que $G \in L$.
- Si todos los vértices tienen grado par, selecciona un vértice no aislado $s \in V$ y ejecuta BFS o DFS para marcar todos los vértices alcanzables desde s.
- Si el número de vértices marcados es menor que el número de vértices no aislados en G (lo que indica que la gráfica es desconexa), la máquina **acepta** (devuelve `1`).
- Si todos los vértices tienen grado par y pertenecen a una sola componente conexa, la gráfica G sí posee un ciclo Euleriano, por lo que la máquina **rechaza** (devuelve `0`).

2. Análisis de Complejidad en Tiempo

- El cálculo de los grados de todos los vértices toma tiempo $O(|V| + |E|)$.
- La comprobación de grados impares requiere recorrer los $|V|$ valores de grado, lo cual toma tiempo $O(|V|)$.
- El algoritmo BFS/DFS para verificar la conectividad de la gráfica se ejecuta en tiempo $O(|V| + |E|)$.

El tiempo total de ejecución es $O(|V|+|E|)$, el cual está acotado por un polinomio respecto al tamaño de la entrada $|\langle G \rangle|$. Por ende:

$$L \in \mathbf{P}$$

Conclusión

Dado que la clase determinista **P** está contenida tanto en **NP** como en **coNP** ($\mathbf{P} \subseteq \mathbf{NP} \cap \mathbf{coNP}$):

Como $L \in \mathbf{P}$, se puede usar el algoritmo determinista M como un verificador trivial que ignora cualquier certificado, lo que demuestra que $L \in \mathbf{NP}$.

Por otra parte, al ser **P** cerrada bajo complemento ($\mathbf{P} = \text{co}\mathbf{P}$), el complemento de L también está en **P**, lo que implica directamente que $L \in \mathbf{coNP}$.

- **HAM_MOD** $= \{\langle G_1, G_2 \rangle \mid G_1$ tiene un ciclo Hamiltoniano y G_2 no tiene un ciclo Hamiltoniano$\}$

Solución

Clasificación del Problema

A diferencia de los otros problemas que se han trabajado en clase directamente o tan siquiera se ha mencionado su respectiva clase de complejidad. Para esta ocasión nos enfrentamos con una modificación en la que si bien parte de un problema que ya conocemos, dicho cambio se realizó con maña para recordar que existen otras clases de complejidad a parte de **P**, **NP** y **coNP**.

Siendo que, decidimos clasificar a HAM_MOD dentro de la clase de tiempo exponencial determinista (**EXP**) de acuerdo a las siguientes razones (la idea nos la dio el profesor al final de la clase del día 27/10/26, simplemente la desarrollamos de una manera más formal y propia):

Por qué NO pertenece a P

Para que un problema pertenezca a la clase **P**, debe existir un algoritmo determinista capaz de decidir cualquier ejemplar en tiempo polinomial. En el caso de HAM_MOD:

– No se conoce ningún algoritmo determinista en tiempo polinomial, propiedad estructural ni teorema eficiente (como el Teorema de Euler para ciclos eulerianos) que permita decidir si una gráfica general posee o no un ciclo Hamiltoniano.

– Dado que el problema del Ciclo Hamiltoniano es un problema emblemático **NP**, determinar si G_1 tiene ciclo y G_2 no lo tiene requeriría resolver dos problemas intratables en un solo paso determinista. Por ende, **no hay evidencia de que HAM_MOD $\in$ P**.

La componente del problema que pertenece en NP (G_1)

La primera condición de la pareja exige que la gráfica G_1 posea un ciclo Hamiltoniano. Esta parte del problema pertenece a la clase **NP**, ya que si la respuesta para G_1 es afirmativa, existe un **certificado corto** (la permutación ordenada de los vértices de G_1) que un verificador determinista puede comprobar en tiempo polinomial $O(|V_1|)$.

La componente del problema que pertenece en coNP (G_2)

La segunda condición exige que la gráfica G_2 **NO** posea ningún ciclo Hamiltoniano. Esta condición pertenece a la clase **coNP** (el complemento de un problema **NP**). Para confirmar que G_2 no tiene ciclos de forma determinista o para descalificar una respuesta falsa, se evalúa con el cuantificador universal ($\forall$), donde se debe garantizar que *ninguna* de las posibles permutaciones de vértices forme un ciclo válido.

Por qué no cabe únicamente en NP ni en coNP

El lenguaje HAM_MOD combina simultáneamente una condición existencial ($\exists$ para G_1) y una condición universal ($\forall$ para G_2) sobre dos instancias independientes. Si quisiéramos clasificar a HAM_MOD dentro de **NP**, necesitaríamos un certificado polinomial para probar que G_2 *no* tiene ciclo Hamiltoniano, lo cual requeriría que **coNP** $\subseteq$ **NP**. De igual forma, si quisiéramos clasificarlo únicamente en **coNP**, requeriríamos que **NP** $\subseteq$ **coNP**. Bajo la conjetura ampliamente aceptada de que **NP** $\neq$ **coNP**, un problema que combina ambas condiciones no puede ser contenido de forma única en ninguna de las dos clases, quedando fuera de la unión de **NP** $\cup$ **coNP**.

Conclusión del análisis

Dado que tanto **NP** como **coNP** están contenidas en la clase de tiempo exponencial determinista (**NP** $\subseteq$ **EXP** y **coNP** $\subseteq$ **EXP**), cualquier problema definido por este tipo de condiciones se puede **decidir mediante un algoritmo determinista de fuerza bruta**.

P.D. que HAM_MOD pertenece a EXP mediante una máquina determinista decidible

1. **Construcción del Decisor Determinista $M((G_1, G_2))$:**

- **Fase 1 (Verificar G_1):** Explora de forma determinista las $|V_1|!$ posibles permutaciones de vértices de G_1. Si encuentra al menos una permutación que forme un ciclo Hamiltoniano válido, continúa con la fase 2; si explora todas sin éxito, **Rechaza**.
- **Fase 2 (Verificar G_2):** Explora de forma determinista las $|V_2|!$ posibles permutaciones de vértices de G_2. Si comprueba que **ninguna** permutación forma un ciclo Hamiltoniano, **Acepta**; si encuentra alguna que sí forma ciclo, **Rechaza**.

2. **Análisis de Complejidad de Tiempo:**
   La exploración de G_1 toma tiempo $O(|V_1|! \cdot |V_1|) = O(2^{c_1|V_1|\log|V_1|})$.

   La exploración de G_2 toma tiempo $O(|V_2|! \cdot |V_2|) = O(2^{c_2|V_2|\log|V_2|})$.

   El tiempo total del algoritmo es la suma de ambos tiempos exponenciales:

   $$T(n) = O(2^{n^k})$$

   donde $n = |\langle G_1, G_2\rangle|$ es el tamaño de la entrada.

Conclusión final

Dado que no existe un algoritmo de tiempo polinomial determinista conocido, y al combinar dos condiciones asimétricas (**NP** y **coNP**) sobre instancias distintas que pueden resolverse exhaustivamente en tiempo exponencial, se conjetura que la clase de complejidad adecuada para HAM_MOD es **EXP**.

- **CONECTIVIDAD** $= \{\langle G\rangle \mid G$ es una gráfica conexa$\}$

Solución

Definición Formal del Lenguaje CONECTIVIDAD

El lenguaje del problema se define en su versión de decisión como:

$$\text{CONECTIVIDAD} = \{\langle G\rangle \mid G \text{ es una gráfica no dirigida y conexa}\}$$

Una gráfica no dirigida $G = (V, E)$ es **conexa** si para todo par de vértices $u, v \in V$, existe al menos un camino (ruta) en G que conecta a u con v.

Clasificación de Complejidad

El problema **CONECTIVIDAD pertenece a la clase P**.

Dado que se cuenta con algoritmos deterministas de exploración de gráficas que pueden verificar la conexidad en tiempo lineal respecto al número de vértices y aristas, el problema es decidible en tiempo polinomial determinista.

P.D. que CONECTIVIDAD pertenece a P mediante una máquina determinista decidible

1. Algoritmo Determinista de Decisión $M(\langle G \rangle)$

Dada la codificación estándar de una gráfica no dirigida $G = (V, E)$ (por ejemplo, mediante listas de adyacencia):

- Si $0 \leq |V| \leq 1$, la gráfica es trivialmente conexa, por lo que la máquina **Acepta** (devuelve `1`).
- Selecciona un vértice inicial arbitrario $s \in V$.
- Inicializa un arreglo de marcas o un conjunto de nodos visitados $S = \{s\}$.
- Ejecuta BFS o DFS comenzando en s:
  - Mientras existan nodos no visitados, toma un nodo $u \in S$ no procesado, examina todos sus vecinos v en G y los añade al conjunto S.
- Compara la cardinalidad del conjunto de nodos visitados $|S|$ con el total de vértices $|V|$.
- Si $|S| = |V|$ (es decir, todos los nodos de G fueron alcanzados desde s), la máquina **Acepta** (devuelve `1`).
- Si $|S| < |V|$ (existen nodos inalcanzables desde s), la máquina **Rechaza** (devuelve `0`).

2. Análisis de Complejidad de Tiempo

La inicialización toma tiempo $O(|V|)$.

El algoritmo de exploración (BFS/DFS) visita cada vértice a lo más una vez y examina cada arista dos veces (una por cada uno de sus extremos en una gráfica no dirigida).

Por lo tanto, el tiempo total de ejecución está acotado por:

$$T(n) = O(|V| + |E|)$$

Como $|V|+|E|$ es proporcional a la longitud de la cadena de entrada $n = |\langle G \rangle|$, la función de complejidad de tiempo es polinomial $O(n)$. Por ende:

$$\text{CONECTIVIDAD} \in \mathbf{P}$$

Conclusión

Al ser un lenguaje perteneciente a la clase determinista **P**:

Como $\mathbf{P} \subseteq \mathbf{NP}$, el algoritmo determinista M actúa como un verificador que ignora cualquier certificado, manteniendo la complejidad de tiempo $O(|V| + |E|)$.

Por otro lado, como la clase **P** es cerrada bajo complemento ($\mathbf{P} = \text{co}\mathbf{P}$), el complemento $\overline{\text{CONECTIVIDAD}}$ (las gráficas desconexas) también pertenece a **P**, concluyendo que **CONECTIVIDAD** $\in$ **coNP**.

- **(*) CONJ_INDEP** $= \{\langle G, k\rangle \mid$ la gráfica G tiene un conjunto independiente de tamaño al menos $k\}$
  ** Un conjunto independiente de los vértices de una gráfica G es un conjunto tal que ninguno de sus vértices es adyacente a otro.*

Solución

- **(*) K-COLORACIÓN** $= \{\langle G\rangle \mid G$ tiene una coloración de sus vértices, con K colores$\}$
  Investiga qué es una coloración en gráficas.

Solución

- **(*) EC-CUADRATICAS** $= \{\langle a, b, c\rangle \mid a, b, c \in \mathbb{Z}^+, \exists x, y \in \mathbb{Z}^+$ tal que $ax^2 + by = c\}$

Solución

** Opcional para equipos de 3 o menos integrantes. Equipos de 4 integrantes es obligatorio.*

Ejercicio 3

Enunciado

Considera el siguiente acertijo.
Bruno, Juan y Samuel trabajan en una ciudad como bombero, policía y maestro, aunque no necesariamente en ese orden.
En la ciudad se rumora lo siguiente:
- Bruno y el maestro son vecinos
- Juan y el maestro son vecinos
- Bruno y Samuel (ambos) son vecinos del bombero
- El policía y el bombero (ambos) son vecinos de Juan
- Todos son vecinos entre sí

Se sabe que solo dos de estas afirmaciones son verdaderas. ¿Qué oficio desempeña cada uno?

1. Expresa el acertijo anterior como un ejemplar concreto de algún problema en NP; Enuncia el problema en su versión canónica.
   Nota: Si el problema elegido no es uno de los vistos en clase, demuestra que el problema está en NP.

Solución

2. Da una representación visual (diagrama, figura, imagen, etc..) del ejemplar concreto, de acuerdo al problema del inciso anterior.

Solución

3. Resuelve el acertijo exhibiendo un certificado.

Solución

Ejercicio 4

Enunciado

Para cada uno de los lenguajes del ejercicio 2:
- Define el lenguaje correspondiente al complemento.
- ¿A qué clase de complejidad pertenece el complemento?
- Da un ejemplar concreto que pertenezca al complemento.

- **Problema DOM:**

Solución

- **Problema L (No ciclos Eulerianos):**

Solución

- **Problema HAM_MOD:**

Solución

- **Problema CONECTIVIDAD:**

Solución

- **Problema CONJ_INDEP (Opcional *):**

Solución

- **Problema K-COLORACIÓN (Opcional *):**

Solución

- **Problema EC-CUADRATICAS (Opcional *):**

Solución